

PROJECT REPORT

Operate

A self-hosted company management system for a plywood and flush-door manufacturer — order book, employee work tracking and order traceability in one application.

DEVELOPER

Gopi Patel

REPORT DATE

25 September 2026

ENGAGEMENT

Client project — in progress

CLIENT

Plywood and flush-door manufacturer

SCOPE OF THIS REPORT

Build 1 — web application

STATUS

**Phases 1–4 delivered · 5–10 in progress
or pending**

STACK

NestJS · Prisma · PostgreSQL · React

VERIFIED TESTS

127 passing (108 API, 19 web)

Started 21 August 2026 · development continuing

Private client project · not for redistribution

Contents

1	Project Overview	12	AI-Assisted Development
2	Objectives	13	Challenges and Solutions
3	Problem and Requirements	14	Testing and Verification
4	Solution Overview	15	Security, Reliability, Data Integrity
5	Features and Current Status	16	Deployment and Usage
6	System Architecture	17	Role and Contribution
7	Technology Stack	18	Verified Metrics
8	Core Workflows	19	Limitations
9	Engineering Decisions	20	Future Improvements
10	Data Model and API Surface	21	Conclusion
11	Implementation Details	22	Master Prompt

How to read this report

Every number, behaviour and status in this document was read from the working repository, the running application or its test output on 25 September 2026. Where something is designed but not built, it is labelled **Planned** rather than described in the present tense. Known defects are stated in **§19 Limitations** rather than omitted.

THIS IS AN ACTIVE CLIENT PROJECT

Operate is being built for a client — a plywood and flush-door manufacturer — and the engagement is ongoing. Requirements come from the client, are confirmed before implementation, and continue to arrive: several features described here, including the dedicated order-capture account and the Boss App section, were added mid-build at the client's request. This report therefore describes the system as it stood on 25 September 2026, not a finished product. Scope beyond Phase 4 will be shaped by the client's decisions as they are made.

TERMINOLOGY

Chalan — the delivery/challan document that accompanies goods. **ERP** — in this project, the employee work module: tasks, submissions and (from Phase 7) wages. **Order Book** — the record of customer orders from arrival to completion. **Build 1/2/3** — the three delivery stages of the product: web application, Android application, and home-server deployment.

1 · Project Overview

Operate is a management system built for a client: a small plywood and flush-door manufacturing business. The client receives customer orders informally, produces goods against them, and dispatches them with paperwork; the people doing the work are paid for what they complete. Until the project began, those three facts lived in separate places — messages on a phone, notes on paper, and somebody's memory — with no single record tying them together.

The work is delivered in phases against requirements agreed with the client, and the engagement is **ongoing**: four of ten phases are delivered and in use for testing, and the remaining scope is being specified with the client as the earlier phases are reviewed.

The system is built as **one application with three modules**, joined by a single permanent order number:

MODULE	WHAT IT IS RESPONSIBLE FOR	STATE
Order Book	Customer orders from arrival to completion: capture of the original message, correction, approval, and allocation of the permanent order number every other module refers to.	Built
ERP	Employee work: tasks created and assigned by the administrator, started and submitted by the employee with evidence, reviewed and accepted or returned for correction. Wages are computed from accepted work.	Tasks built, wages planned
Order Traceability	A sequential chain of custody for each order — chalan, manufacturing, sales chalan, dispatch photograph, gate pass — where each stage may only be entered after the one before it, and mismatches between stages are detectable.	Planned (Phase 5)

Intended users

USER	WHAT THEY DO IN THE SYSTEM
Owner / administrator the client	Approves orders, creates and reviews tasks, manages employee accounts and their permissions, and sees the company-wide dashboard and activity log.
Order-capture employee	Pastes incoming customer messages into the system so they become pending orders. Has no access to the Order Book itself.
Factory employee	Sees only their own assigned work, starts it, and submits it with whatever evidence the task requires — a count, a note, a photograph.
Boss	A read-only supervisory account with its own section: orders, ERP and chalans. The three sections are scaffolded against real data and await their detailed specification.

Delivery in three builds

The product is planned as three builds over the same backend and database, so that no data is duplicated and no business rule is implemented twice:

- **Build 1 — web application.** The subject of this report. Runs on one ordinary PC, reachable from the local machine.
- **Build 2 — Android application (Flutter).** The same API, the same database, the same permissions. **Planned**
- **Build 3 — home server.** The owner's computer becomes the central server reachable from outside. The database is never exposed directly; when the server is unreachable, clients say so rather than falling back to a

local copy that could diverge. **Planned**

COST CONSTRAINT

A hard requirement set by the client is that the system must run at no additional cost: every component is free and open source, self-hosted on hardware the client already owns. There is no cloud service, no paid API and no subscription anywhere in the system — including in development tooling, which is a portable toolchain installed under a local directory rather than system-wide.

2 · Objectives

OBJECTIVE	HOW IT IS EXPRESSED IN THE BUILD
One system, one source of truth	A single database and a single backend own all business logic. Clients — web today, Android later — hold no rules of their own.
Authorization enforced by the server	Hiding a button is not access control. Every route is closed unless it declares otherwise, every action requires a named permission, and every query is narrowed to the rows the caller is entitled to.
An order number that is never reused	Allocation is a single atomic database statement, so simultaneous approvals cannot collide. Every other module keys off that number.
Everything important is audited	Successful actions, refusals and authentication events are all recorded, in a log the application itself cannot rewrite.
Work is evidenced, not asserted	A task states what finishing it requires; submissions carry that evidence; acceptance is a separate act by a second person.
Zero running cost, self-hosted	Open-source components only, on one PC, with no external service in the request path.
Business rules are asked for, not invented	Where the specification was silent — how quantities are expressed, whether an order may ship in parts, what happens to a returned submission — the answer was obtained from the client before implementation and then encoded.
Delivered in verifiable phases	Ten phases, each ending in a working system with its own tests, rather than one long build with a single integration at the end.

3 · Problem and Requirements

3.1 The problem

Orders reach the client as free-form messages: a customer name, then lines such as 50 Flush Door 7x3 30mm , written differently by every customer and sometimes ambiguously by the same one. They are acted on immediately and recorded later, if at all. The consequences are ordinary but expensive: an order worked on twice, a dispatch that cannot be matched to a chalan, a dispute about what was ordered with no evidence either way, and wages calculated from memory at the end of the month.

The requirement is therefore not merely to store orders. It is to make the record *authoritative*: to keep the customer's original words next to the system's reading of them, to make each stage of an order follow the previous one, and to make every change attributable to a person.

3.2 Functional requirements

AREA	REQUIREMENT	STATUS
Order intake	Accept an order message, read what it can, and keep the original text verbatim.	Built
Order approval	An administrator corrects and approves; approval issues the permanent order number.	Built
Customers & products	Identify a repeat customer; build a product catalogue from what is actually ordered.	Built
Employee tasks	Create, assign, start, submit with evidence, review, return for correction.	Built
Photographic evidence	Upload photographs from a phone; store them safely; serve them only to those permitted.	Built
Accounts	Administrator creates employee accounts, sets and changes credentials, and removes accounts.	Built
Audit	Record every significant action and every refusal, tamper-evidently.	Built
Traceability chain	Five sequential stages per order, each blocked until the previous is complete.	Phase 5

AREA	REQUIREMENT	STATUS
Mismatch detection	Detect contradictions between stages — for example the same photograph used for two orders.	Partly: identical photographs are already detectable by content hash
Wages	Monthly wage calculation from accepted work at the rate agreed when the task was created.	Phase 7
WhatsApp intake	Orders arriving directly from WhatsApp rather than being pasted in.	Endpoint and parser built; no provider connected

3.3 Non-functional requirements and constraints

CONSTRAINT	CONSEQUENCE FOR THE BUILD
No additional cost	Open-source components only; no managed database, no cloud storage, no paid APIs.
Self-hosted on one PC	Runs on a dual-core desktop with 8 GB of memory; image processing and password hashing are tuned to that, not to a server.
Database never exposed	PostgreSQL listens on the loopback interface only; the API is the sole process that connects to it.
Passwords never stored in plain text	Argon2id hashing; passwords are never written to the audit log or any other record.
No hard-coded secrets or permissions	Secrets come from an environment file that is never committed; permissions are database rows, editable without a redeployment.
Windows, no administrator rights	No Docker and no system services: a portable Node.js and PostgreSQL under a local directory, started by scripts in the repository.
Phone-shaped screens matter	The interface is usable at 375 px; tap targets are at least 44 px tall.
No internet in the request path	No web fonts and no CDN — everything is served from the origin, because Build 3 may run on a local network with no internet access.

4 · Solution Overview

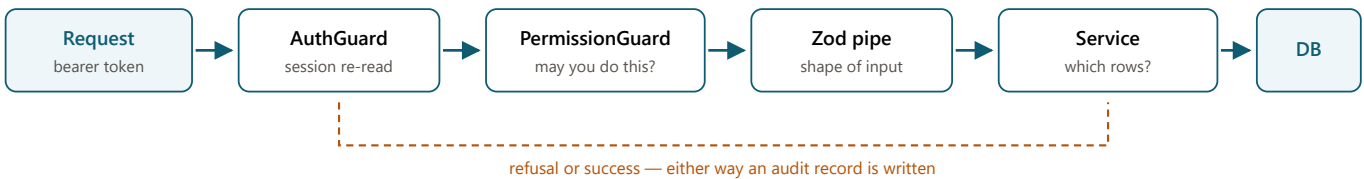
Operate is a TypeScript monorepo containing three packages: a NestJS API that owns the database and all business rules, a React web client, and a shared contracts package holding the request and response shapes, the permission catalogue and the role definitions used by both sides.

PACKAGE	RESPONSIBILITY	LINES
apps/api	NestJS on Fastify. The only process that touches PostgreSQL or the file store. Guards, business services, audit, migrations and seed.	5,458
apps/web	React single-page client: administrator console, employee screens, order capture, Boss App.	5,740
packages/contracts	Zod schemas, permission keys, role definitions and shared types, published to both sides in CommonJS and ES module form.	997
apps/api/test	Tests that run against the real application and the real database.	1,838
apps/api/prisma	Schema, seven migrations, and an idempotent seed.	1,922
scripts	Database control, environment bootstrap, secret generation.	252

Line counts are of tracked files, excluding the dependency lockfile; 17,497 lines in total across 134 tracked files.

The shape of a request

Every request takes the same path, and each step can only narrow what the one before it allowed:



What exists today

37 HTTP ENDPOINTS	22 DATABASE TABLES	43 PERMISSIONS	127 PASSING TESTS
-----------------------------	------------------------------	--------------------------	-----------------------------

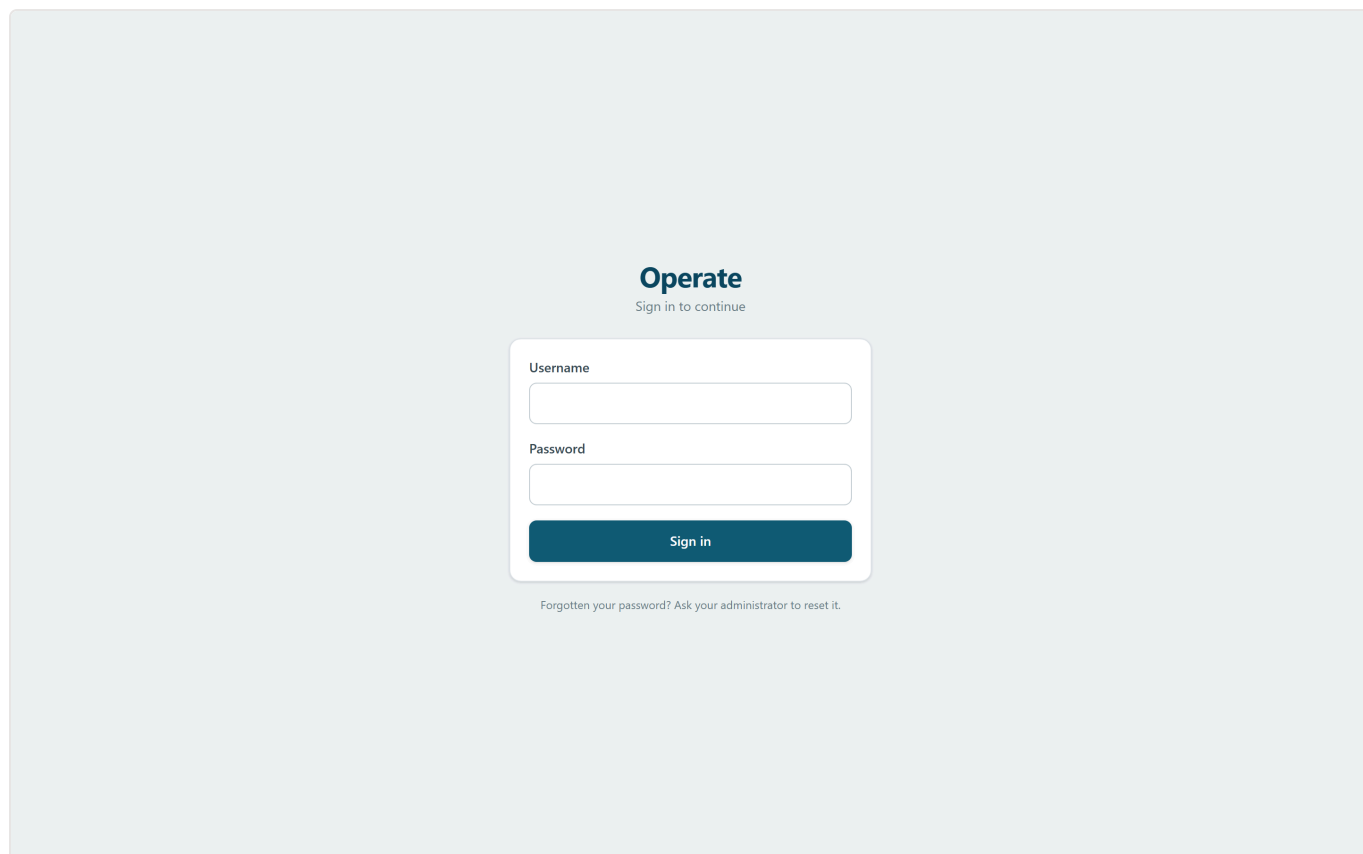


Figure 1 — Sign-in. Repeated failures lock the account for a period; the message never reveals whether the username exists.

5 · Features and Current Status

The project is delivered in ten phases. Four are complete and with the client for testing, one is partly complete, and the rest are either specified but not started or waiting on decisions the client has still to make. The table below is the state of the build on 25 September 2026.

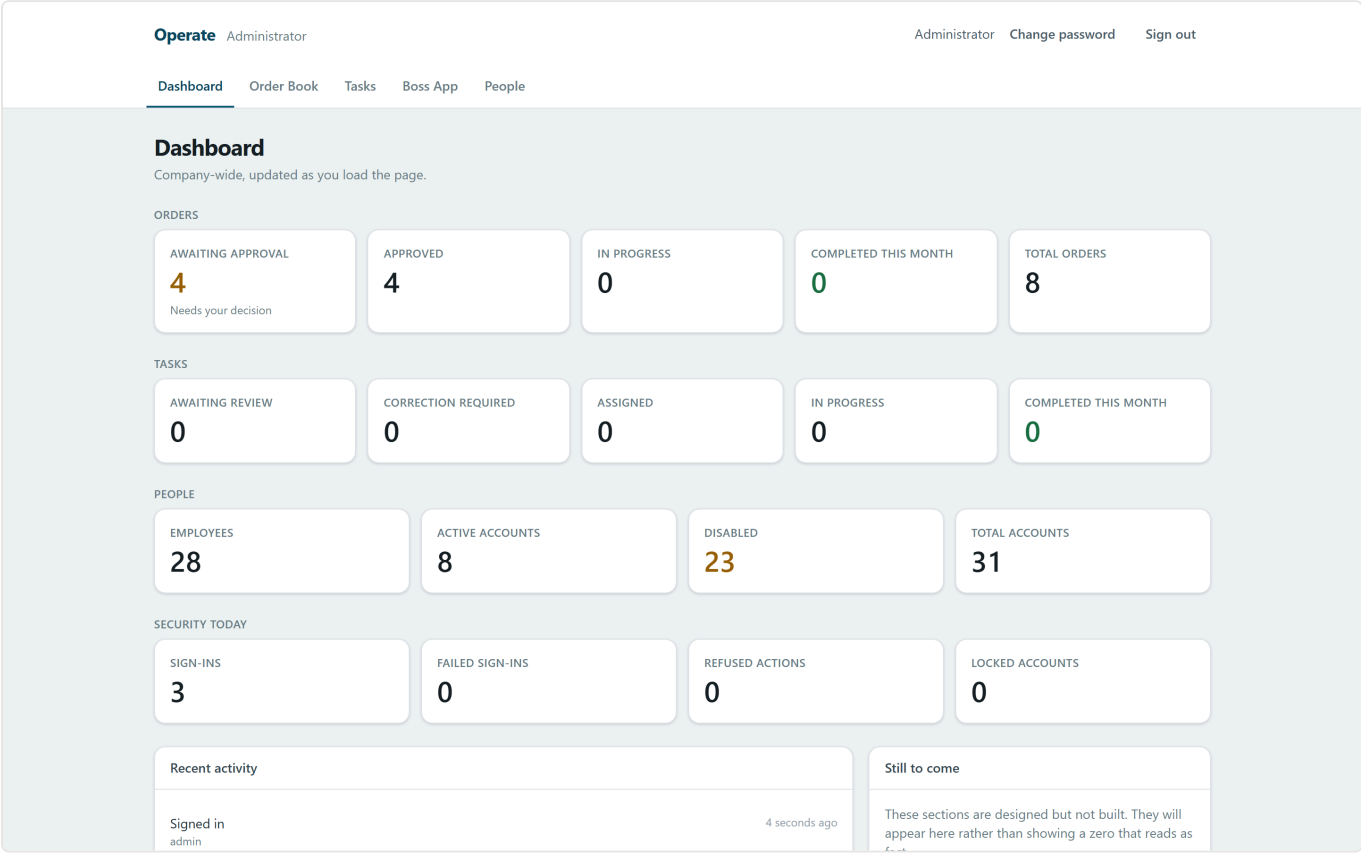


Figure 2 — The administrator dashboard: orders, tasks, people and the day's security events. "Still to come" names the modules that are not built yet and the phase each arrives in, rather than reporting them as zero. Counts are from the development database.

PHASE	SCOPE	WHAT WORKS TODAY	STATUS
1	Database, authentication, authorization, audit	Sign-in with logout, rotating refresh tokens, deny-by-default routing, permission and row-level checks, append-only hash-chained audit log.	Done
2	Application shell and dashboards	Permission-filtered navigation, administrator dashboard (orders, tasks, people, security, recent activity), employee dashboard scoped to the individual.	Done
3	Order Book and intake	Message capture and preview, parser, pending → approved workflow, atomic order numbering, customer matching, catalogue learning on approval.	Done
4	ERP tasks and file storage	Task creation and assignment, per-task evidence requirements, start/submit/review and the correction loop, earnings frozen at creation, photograph upload and permission-checked retrieval.	Done
5	Traceability chain	Not started. Awaiting the client's field-level specification for chalan, manufacturing, sales chalan, dispatch photograph and gate pass.	Awaiting client detail

PHASE	SCOPE	WHAT WORKS TODAY	STATUS
6	Audit and integrity scanning	The log and its hash chain exist and a verification routine is implemented; the scheduled scan and its administrator screen are not built.	Partial
7	Wages and reports	Not started. The data it depends on — accepted submissions and the rate frozen on each task — is already recorded.	Not started
8	Test pass	Ongoing: each phase ships with its own tests (127 today).	Ongoing
9	Interface refinement	Not started as a dedicated pass.	Not started
10	Build 1 acceptance	Not started: includes backup policy and service registration.	Not started

5.1 Capabilities in detail

CAPABILITY	BEHAVIOUR
Message capture	A message is pasted with an optional sender phone number. The parser extracts a customer name and order lines; anything it cannot read is listed for the administrator rather than rejected, and the original text of every line is stored permanently.
Preview before capture	A separate preview endpoint shows exactly what would be read — matched customer, lines, unreadable lines, notes — without creating anything.
Approval	Approval validates that a customer has been chosen and at least one line exists, issues the order number, and records the transition. Rejection and cancellation are separate, recorded actions with their own permissions.
Customer identification	A sender phone number that matches a customer on file links the order automatically; otherwise the name in the message is matched, and failing that the order is captured with no customer and cannot be approved until a person chooses one.
Product catalogue	Products are matched on capture and created only on approval, so a misreading in an unapproved message never becomes a permanent catalogue entry.
Tasks	Each task carries a title, an optional link to an order, an assignee, an optional due date, a rate and three evidence flags — quantity, note, photograph — chosen by whoever creates it.
Submission and review	An employee starts a task and submits the evidence it requires. The administrator accepts it (the task becomes completed) or returns it (the task becomes correction required). Returned submissions are kept; the employee makes a new one.
Photographs	Uploads are re-encoded, resized, stripped of metadata including GPS, and stored under the SHA-256 digest of their contents. Retrieval is permission-checked; there is no public directory.
Account management	The administrator adds an employee (account plus employee record in one transaction), edits the display name, username and password, and removes an account — deleting it outright if it has no history, archiving it if the audit log names it.
Password change	Any signed-in user can change their own password; doing so revokes their other sessions. An administrator cannot change their own password through the account-editing screen — they are directed to the password screen, so the act is always deliberate.

CAPABILITY	BEHAVIOUR
Dashboards	The administrator sees company-wide counts and recent activity; an employee sees only their own work and their own activity. Modules that are not built are named with the phase they arrive in rather than reported as zero.
Boss App	Added mid-build at the client's request: a separate read-only section with three parts — view orders, ERP and chalan — each rendering real data and labelled as awaiting the client's specification of what it should do.

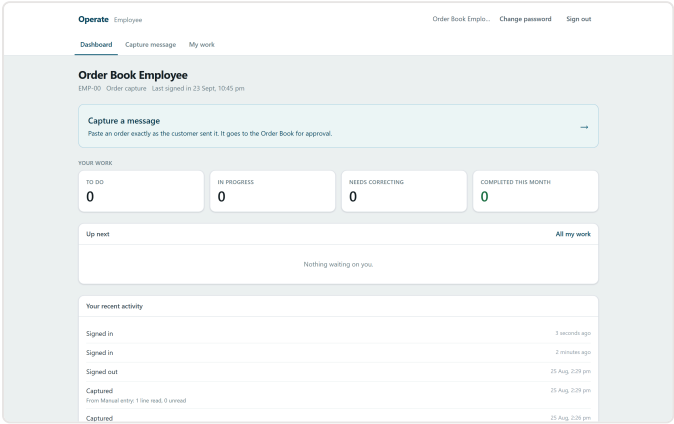


Figure 3 — An employee's home. The person signed in here captures order messages, so that action leads; the counters below are their own work only.

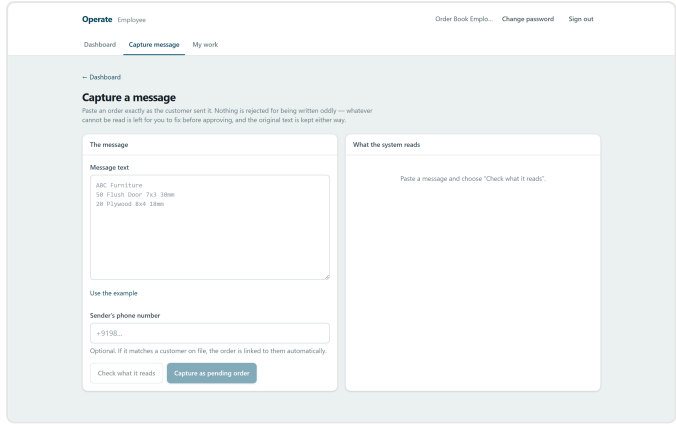


Figure 4 — Capture. The right-hand panel states what the system read, including the lines it could not, before anything is created.

6 · System Architecture

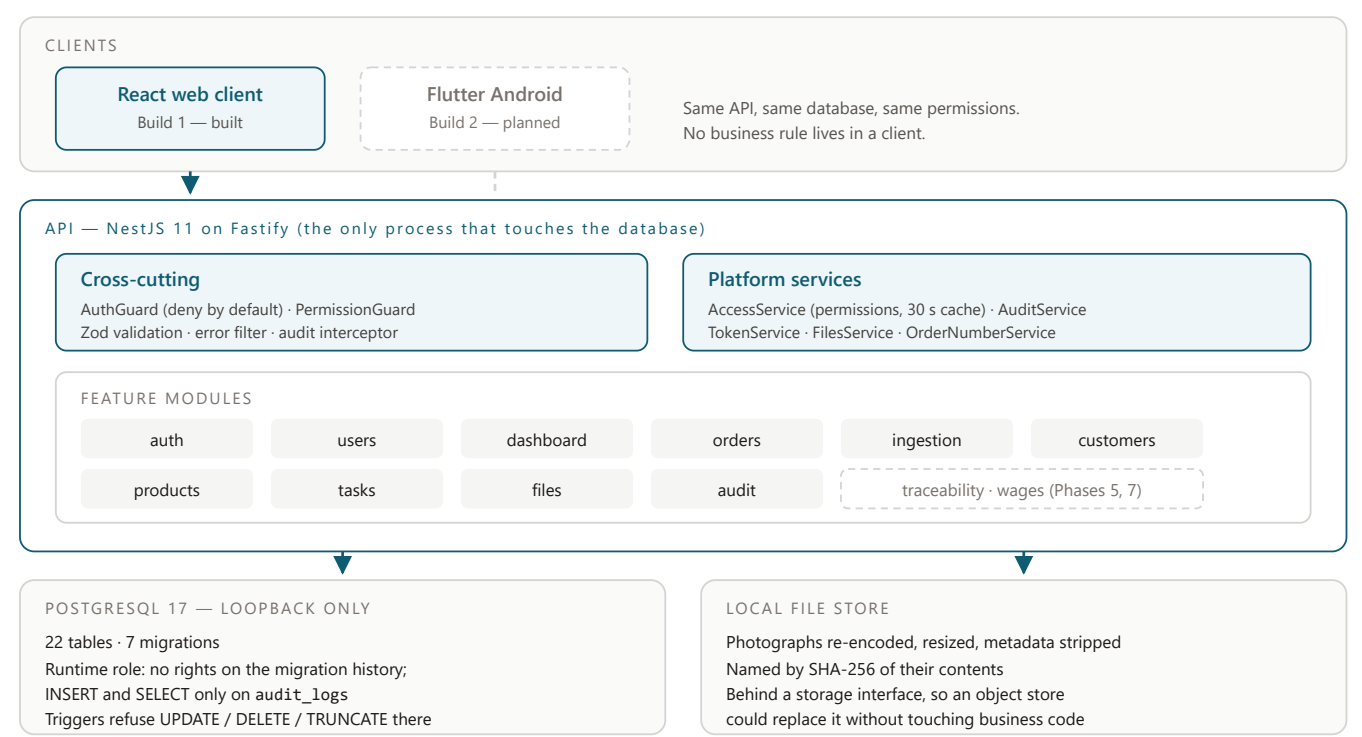


Figure 5 — System architecture. Dashed elements are designed but not built.

6.1 Principles the structure enforces

PRINCIPLE	HOW THE STRUCTURE ENFORCES IT
The API is the only writer	No client, script or report tool connects to PostgreSQL. The database listens on the loopback interface, and the runtime database role owns nothing.
Closed by default	Authentication is a global guard. A route is public only if it is decorated <code>@Public()</code> , so a forgotten decorator leaves a route shut rather than open.
Two authorization questions	A permission answers "may you do this kind of thing"; the service answers "to which rows" by narrowing the query. The second question is the one a permissions-only design cannot ask.
Policy is data	Roles, permissions, role grants and per-user overrides are tables. Even the administrator holds power by enumerated grant — there is no <code>role === 'ADMIN'</code> shortcut anywhere in the guards.
Shared contracts	Permission keys, role definitions, status labels and request schemas live in one package used by both the API and the client, so a permission cannot be spelled differently on the two sides.
Seams where integrations will land	Message intake is an HTTP endpoint with its own source records, not a screen feature; a WhatsApp provider will post to the same route the capture screen uses today.

7 • Technology Stack

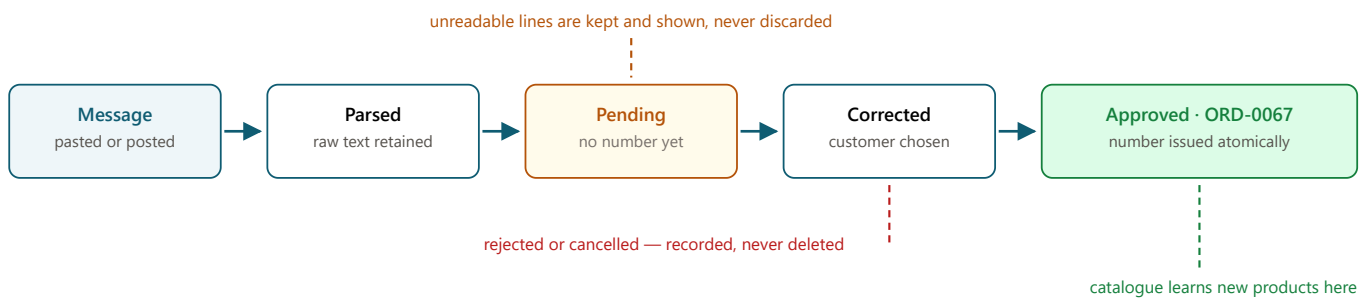
COMPONENT	VERSION	WHY THIS ONE
Node.js	24.19	Portable installation under a local directory — no administrator rights, no system service.
NestJS on Fastify	11	Guards, modules and dependency injection map directly onto "deny by default, then check the permission, then narrow the rows". Fastify keeps the footprint small on an 8 GB machine.
TypeScript	5.7	One language across API, client and shared contracts; the permission catalogue is a typed map, so a misspelled key fails the build.
PostgreSQL	17.7	Table-level privileges, triggers, advisory locks and <code>ON CONFLICT</code> — the features that make the audit log append-only and order numbering atomic.
Prisma	6.19	Typed queries and a migration history that is reviewable as SQL; hand-written migrations sit alongside generated ones where privileges and triggers are involved.
Argon2id (@node-rs/argon2)	2.0	Memory-hard password hashing at the OWASP minimum profile (19 MiB, 2 passes), tuned to the machine it runs on.
Zod	3.24	One schema per request shape, shared with the client, and the same schemas validate the environment file at boot.
sharp	0.35	Re-encodes uploaded photographs: applies rotation, drops metadata, and turns a multi-megabyte phone photograph into a few hundred kilobytes.
React + Vite	19 / 6	A single-page client with fast local iteration; Vite's proxy keeps the browser talking to one origin in development.
TanStack Query	5	Server state, caching and refetching without a bespoke data layer.
Tailwind CSS	4	Utility styling with no component framework to fight; no web fonts, so nothing is fetched from outside.
Jest / Vitest	29 / 3	API tests run against the real application and database; client tests run in jsdom with only the network mocked.

DELIBERATELY ABSENT

No Docker — the target machine has no administrator rights, so the toolchain is portable and started by scripts in the repository. **No cloud services** — no object storage, no managed database, no third-party authentication. **No web fonts or CDN** — Build 3 may run on a network with no internet access, and a font that fails to load must not change how the application reads.

8 · Core Workflows

8.1 An order, from message to number



One row throughout: the pending order and the approved order are the same record, so the original message, every correction and the whole audit trail hang off one identifier.

Order status is a state machine, enforced on the server: `PENDING` → `APPROVED` | `REJECTED` , `APPROVED` → `IN_PROGRESS` | `CANCELLED` , `IN_PROGRESS` → `COMPLETED` | `CANCELLED` . Completed, cancelled and rejected are terminal. Every transition writes a status-history row and an audit record naming the person.

Operate Administrator

AdministratorChange passwordSign out

DashboardOrder BookTasksBoss AppPeople

Order Book

Every order, from the moment it arrives to the moment it ships.

Capture message

All 8Pending approval 4Approved 4In progress 0Completed 0Cancelled 0Rejected 0

Search by order number, customer or product

ORDER	CUSTOMER	CAPTURED	ITEMS	QUANTITY	STATUS
Not yet numbered	ABC Furniture	25 Aug, 2:29 pm	Flush Door	12 pcs	PENDING APPROVAL
Not yet numbered	Patel Traders	25 Aug, 2:26 pm	Flush Door + 1 more	45 pcs	PENDING APPROVAL
Not yet numbered	ABC Furniture	25 Aug, 1:43 pm	doors + 2 more	24 pcs	PENDING APPROVAL
ORD-0067	ABC Furniture	21 Aug, 10:28 pm	Flush Door + 1 more	—	APPROVED
Not yet numbered	Patel Traders	21 Aug, 10:19 pm	Laminate Sheet	40 sheets	PENDING APPROVAL
ORD-0003	ABC Furniture	21 Aug, 10:19 pm	Flush Door + 1 more	—	APPROVED
ORD-0002	Patel Traders	21 Aug, 10:19 pm	Plywood	100 sheets	APPROVED
ORD-0001	ABC Furniture	21 Aug, 10:19 pm	Flush Door + 1 more	—	APPROVED

Figure 6 — The Order Book. Approved orders carry their permanent number; pending ones say plainly that they do not have one yet. The status filters count what is actually there, and capture is one click away.

8.2 A task, from assignment to acceptance

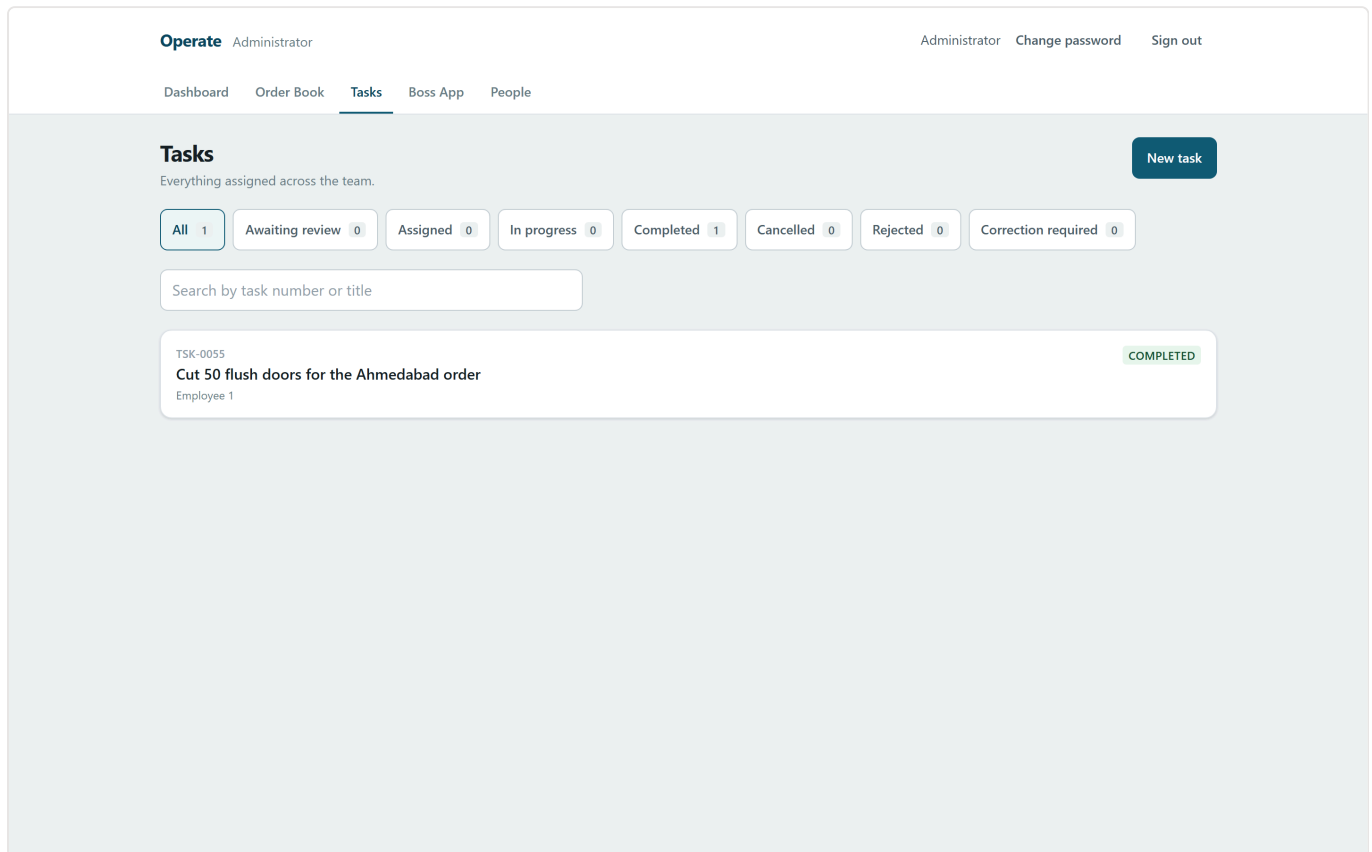
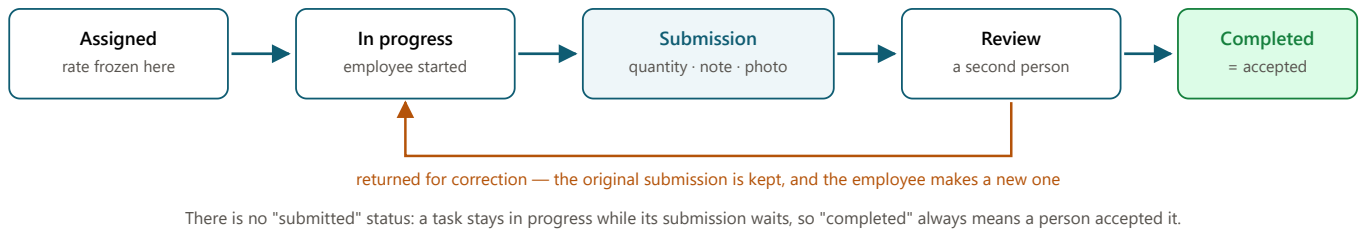


Figure 7 — Tasks. An administrator sees everyone's work and can filter by state; an employee opening the same route sees only the tasks assigned to them, because the query itself is narrowed on the server.

8.3 Signing in and staying signed in

1. Credentials are checked with Argon2id. A username that does not exist is verified against a decoy hash, so a wrong username and a wrong password take the same time and give the same message.
2. Repeated failures lock the account for a configured period; every attempt, successful or not, is recorded.
3. Success returns a short-lived access token, held in a JavaScript variable rather than storage, and a refresh token that is stored only as a hash and returned in an `httpOnly`, `SameSite=strict` cookie.
4. Refresh rotates the token. A token presented twice is treated as theft: the whole session is revoked. The client therefore refreshes single-flight, so a screen firing four queries on one expired token cannot revoke its own session.
5. Sessions are re-read from the database on every request, so disabling an account ends access immediately rather than when the access token happens to expire.

9 · Engineering Decisions

Each decision below is stated as the problem it answers, what was decided, how it is implemented, and what it produced.

9.1 Order numbers must never be issued twice

PROBLEM	Every module keys off the order number. Two orders sharing one number would corrupt the chain the system exists to protect.
DECISION	Allocate from a counter row in a single atomic statement, and accept gaps.
IMPLEMENTATION	<code>INSERT ... ON CONFLICT DO UPDATE ... RETURNING</code> against a <code>counters</code> row, which PostgreSQL serialises. <code>MAX()+1</code> and <code>COUNT()+1</code> were rejected: both read and write in separate steps.
RESULT	Twelve simultaneous approvals receive twelve distinct numbers — asserted by a test. A number is consumed even if the surrounding transaction rolls back: a gap is a curiosity, a reused number is a corrupted chain.

```
// apps/api/src/modules/orders/order-number.service.ts
INSERT INTO counters (key, "nextValue", "updatedAt")
VALUES (${key}, 2, now())
ON CONFLICT (key) DO UPDATE
  SET "nextValue" = counters."nextValue" + 1, "updatedAt" = now()
RETURNING counters."nextValue" - 1 AS allocated
```

9.2 The audit log must survive the application being wrong

PROBLEM	An audit log that the application can rewrite proves nothing — including about the application itself.
DECISION	Enforce append-only in the database, not in code, and make edits made outside the application detectable.
IMPLEMENTATION	The runtime database role holds <code>SELECT</code> and <code>INSERT</code> on <code>audit_logs</code> and nothing else; triggers refuse <code>UPDATE</code> , <code>DELETE</code> and <code>TRUNCATE</code> even from the table's owner. Each row carries a SHA-256 over its contents chained to the previous row's hash, written under a transaction-scoped advisory lock so the order is definite.
RESULT	2,900 audit records exist in the development database, of which 291 are refusals. Removing the protection requires dropping a trigger — itself a schema change, and therefore visible in a diff. (A defect in chain verification found while preparing this report is described in §19.)

9.3 Authorization is asked twice, in two different ways

PROBLEM	"Employee 1 must not read employee 2's data" is not a question a permission can answer: both employees hold the same permission.
DECISION	Separate the kind of action from the set of rows, and apply the second check by narrowing the query rather than filtering results.
IMPLEMENTATION	A guard checks the named permission. The service then builds its <code>where</code> clause from the caller: holders of <code>task.read.any</code> see everything, everyone else is confined to their own employee id. A request for someone else's task returns the same 404 as a task that does not exist.
RESULT	There is no moment at which another person's row has been loaded, and the endpoint cannot be used to discover which identifiers are real.

```
// apps/api/src/modules/tasks/tasks.service.ts
private scopeFor(auth: AuthContext, requested?: string): Prisma.TaskWhereInput {
  if (auth.permissions.has(P['task.read.any'])) {
    return requested ? { assignedToId: requested } : {};
  }
  if (!auth.employee) return { id: '00000000-0000-0000-0000-000000000000' };
  if (requested && requested !== auth.employee.id) throw AppError.forbidden();
  return { assignedToId: auth.employee.id };
}
```

9.4 Permissions are rows, not constants

PROBLEM	Which employee handles which part of the chain is a business arrangement that changes; a constant in the code would require a redeployment to change it.
DECISION	Roles, permissions, role grants and per-user overrides are database tables, with a revoke beating a grant.
IMPLEMENTATION	43 permissions across 11 groups, three roles, and per-user overrides. The administrator holds power by enumerated grant, so there is one code path for everyone. Re-seeding only ever adds grants — a permission the administrator revoked deliberately is never handed back.
RESULT	Adding the read-only Boss role was four permission rows and a role row, with no change to any guard.

9.5 What finishing a task looks like is data, not code

PROBLEM	Sweeping the yard and producing 40 doors are both "tasks", but proving them means different things.
DECISION	Each task carries three evidence flags — quantity, note, photograph — set by whoever creates it.
IMPLEMENTATION	The server reads the flags from the task, never from the request, and refuses a submission that omits what the task requires. A new kind of work needs a different set of tick boxes, not a new task type.
RESULT	A client cannot talk its way past a requirement by leaving a field out.

9.6 The parser never refuses a customer's order

PROBLEM	Customers write orders however they like. A strict parser would silently lose real business.
DECISION	Read what can be read, set aside what cannot, and keep the original text of every line forever.
IMPLEMENTATION	Capture stores the raw message and each line's original text alongside the system's reading of it, plus notes explaining every guess. A preview endpoint shows all of this before anything is created.
RESULT	When a dispute arises about what was ordered, the evidence is what the customer actually wrote. 25 parser tests cover the formats seen so far.

9.7 Guessing a customer is worse than admitting we do not know

PROBLEM	A message may not identify its sender at all.
DECISION	Capture the order with no customer, say so plainly, and refuse approval until a person chooses one.
IMPLEMENTATION	The customer link is nullable; the phone number is tried first, then the name; approval validates that a customer exists.
RESULT	A numbered order always belongs to somebody, which every downstream module depends on.

9.8 Photographs are content-addressed and never publicly served

PROBLEM	Phone photographs are large, carry GPS metadata, and are evidence — the same photograph re-used for two orders is one of the mismatches the system is meant to catch.
DECISION	Re-encode every upload and store it under the SHA-256 digest of its processed contents.
IMPLEMENTATION	Rotation is applied, the image is resized to a maximum dimension, metadata is dropped, and the result is written under its digest behind a storage interface. Every read goes through a permission check; no directory is exposed by the web server.
RESULT	A multi-megabyte photograph becomes a few hundred kilobytes, and duplicate evidence is detectable by storage design rather than by a special rule.

9.9 Deleting an account must not erase the record

PROBLEM	The administrator needs to remove accounts, but the audit log names people, and a record that loses its subject is worth less.
DECISION	Delete an account that has no history; archive one that has.
IMPLEMENTATION	The audit log's reference to a user is <code>ON DELETE RESTRICT</code> , so the database itself refuses to lose the link. The service checks for history, then either deletes the row or sets it archived and inactive, revoking its sessions. The screen explains which will happen before it happens.
RESULT	Removal is never destructive by accident, and an archived person still resolves everywhere the log names them.

9.10 Tokens never touch browser storage

PROBLEM	A token in <code>localStorage</code> is readable by any injected script and outlives the tab.
DECISION	Keep the access token in a module variable and the refresh token in an <code>httpOnly</code> cookie; refresh single-flight.
IMPLEMENTATION	Reloading the page loses the access token and silently restores the session from the cookie. Because refresh tokens rotate and re-use is treated as theft, concurrent refreshes share one in-flight request.
RESULT	Rotation stays safe on a busy page, and no token is readable by script.

9.11 Intake is a seam, not a feature

PROBLEM	WhatsApp intake is wanted, but no provider is connected and none may be free.
DECISION	Define the contract now and stand in for the transport.
IMPLEMENTATION	<code>POST /ingest/messages</code> takes a message, its source and an external identifier used for de-duplication. Today the only caller is a person pasting into a screen; a provider will post to the same route, through the same parser, into the same approval.
RESULT	The integration, when it lands, changes no part of the Order Book. 650 raw messages have already passed through this path in development.

9.12 The interface tells the truth when the server is gone

PROBLEM	Build 3 requires that clients never fall back to a local copy of the data. A vague error invites the user to assume the data is simply empty.
DECISION	Distinguish "the API refused" from "the API could not be reached", and say the second one plainly.
IMPLEMENTATION	Every error the API raises carries a JSON body with an error code; a 5xx without one, or a request that never completes, is reported as <i>Server offline</i> . Three client tests pin this behaviour.
RESULT	A user whose server is down is told so, rather than being shown an empty Order Book.

10 · Data Model and API Surface

10.1 Tables

22 tables across seven migrations. Table names are snake_case; primary keys are UUID v7, so identifiers sort by creation time without exposing a guessable sequence.

GROUP	TABLES	NOTES
Identity & access	users , roles , permissions , role_permissions , user_permission_overrides	Policy is data. An override can revoke as well as grant, and a revoke wins.
Sessions	sessions , refresh_tokens , login_attempts	Refresh tokens are stored hashed; attempts feed the lockout and the security counters.
People	employees	Employee code, designation and wage rate, separate from the login account.
Record	audit_logs	Append-only by database privilege and trigger; hash-chained; references a user with ON DELETE RESTRICT .
Order Book	orders , order_items , order_status_history , customers , products , counters	Order number is nullable until approval; items keep the customer's original text; quantities are decimal with a unit.
Intake	ingestion_sources , raw_messages	A message is a record in its own right, de-duplicated by external identifier.
ERP	tasks , task_submissions , task_events , attachments	Submissions are history: a returned one is kept and a new one made.

10.2 Permission catalogue

GROUP	KEYS	EXAMPLES
users	8	user.read.any , user.read.self , user.create , user.update , user.delete , user.password.reset
tasks	8	task.read.any , task.read.self , task.create , task.assign , task.submit , task.review
orders	7	order.read.any , order.approve , order.reject , order.cancel , ingest.submit
boss	4	boss.app.access , boss.orders.view , boss.erp.view , boss.chalan.view
access	4	role.read , role.manage , permission.read , permission.assign
employees	3	employee.read.any , employee.read.self , employee.manage
customers · products · files · audit · dashboard	9	customer.manage , product.manage , file.upload , file.read.any , audit.read.self , dashboard.read.admin

ROLE	DEFAULT GRANTS
Administrator	All 43 permissions.

ROLE	DEFAULT GRANTS
Employee	Six: read own account and employee record, read own audit entries, read and submit own tasks, upload files. Anything further is granted individually.
Boss	Ten: the four Boss App permissions, read-any on orders and tasks, plus the three self-scoped reads and task read-self.

59 role-permission rows in the running database, plus per-user overrides — for example the order-capture account, which is an ordinary Employee with `ingest.submit` granted individually.

10.3 API surface

37 endpoints under `/api/v1`. Every one requires authentication except the health check and sign-in.

ENDPOINT	PERMISSION	PURPOSE
POST <code>/auth/login</code>	public	Sign in; issues access and refresh tokens.
POST <code>/auth/refresh</code>	public (cookie)	Rotate the refresh token; re-use revokes the session.
POST <code>/auth/logout</code> , POST <code>/auth/password</code> , GET <code>/auth/me</code>	signed in	End a session, change own password, read own identity and permissions.
GET <code>/dashboard/admin</code> · GET <code>/dashboard/me</code>	<code>dashboard.read.admin</code> · self	Company-wide counts; personal counts and next work.
GET/POST/PATCH/DELETE <code>/users</code>	<code>user.*</code>	List, create, edit and remove accounts.
GET/POST/PATCH <code>/orders</code> , POST <code>/orders/:id/{approve,reject,cancel}</code>	<code>order.*</code>	The Order Book and its state transitions.
POST <code>/ingest/preview</code> · POST <code>/ingest/messages</code>	<code>ingest.submit</code>	Read a message without storing it; capture one as a pending order.
GET/POST/PATCH <code>/customers</code> · GET <code>/products</code>	<code>customer.*</code> , <code>product.read</code>	Customer records and the learned catalogue.
GET/POST/PATCH <code>/tasks</code> , POST <code>/tasks/:id/{start,submit,review,cancel}</code>	<code>task.*</code>	The ERP task lifecycle.
POST <code>/files</code> · GET <code>/files/:id</code>	<code>file.upload</code> · <code>permission-checked</code>	Upload a photograph; retrieve one.
GET <code>/health</code>	public	Process and database reachability.

11 · Implementation Details

11.1 Repository layout

apps/api/	NestJS backend – the only thing that touches the database
prisma/	schema, 7 migrations, idempotent seed
src/common/	guards, decorators, error filter, Zod pipe, UUID v7
src/modules/	auth · access · audit · users · dashboard · orders · ingestion · customers · products · tasks · files
test/	security · orders · tasks · ingestion · audit
apps/web/	React client
src/components/	layout, shared interface primitives, authenticated image
src/features/	auth · dashboard · orders · tasks · people · boss
src/lib/	API client, auth context, queries, formatting
packages/contracts/	Zod schemas, permission keys, role definitions (CJS + ESM)
scripts/	database control, environment bootstrap, secret generation
start.cmd / stop.cmd	one-command start and stop on Windows

11.2 Details worth noting

AREA	IMPLEMENTATION
Configuration	The environment is validated by a Zod schema once at boot; a missing or malformed secret stops the process starting. The JWT secret has a minimum length, and the bootstrap script generates one rather than shipping a default.
Two database roles	Migrations run as the owner; the API connects as a runtime role that owns nothing and cannot read the migration history. New tables inherit the runtime grants automatically through default privileges, so a later phase cannot ship a table the API cannot use — or be "fixed" by widening its rights.
Error handling	A single application error type carries a stable code, a human message and optional field errors; one filter turns it into a response. Clients branch on the code and display the message.
Audit writes never fail a request	If writing the record fails, the business action still succeeded, so the failure is logged loudly for the operator rather than turned into a 500 that would lose the action as well as the record.
Permission caching	Effective permissions are cached for 30 seconds per user, so a request does not re-read the grant tables repeatedly; sessions themselves are always re-read.
Authenticated images	A plain cannot attach a bearer token, so photographs are fetched as blobs by a component that holds the credentials and revokes the object URL when it unmounts — rather than weakening the endpoint.
Navigation as one list	Every navigation entry declares the permissions it needs, so "what can this person see" is answerable by reading a single array. It decides only what is shown; the server refuses regardless.
Shared contracts build	The contracts package is compiled twice, to CommonJS for the API and ES modules for the browser, with an exports map — one source of truth for permission keys on both sides.
Interface for small screens	A single scrollable navigation bar rather than a hidden menu, tap targets at least 44 px tall, and tables that become cards below the breakpoint.

Operate Administrator

AdministratorChange passwordSign out

DashboardOrder BookTasksBoss App

People

People

8 accounts. Add someone, or edit anyone to set their name, username and password — then give them those details.

Add employee

NAME	USERNAME	ROLE	CODE	LAST SIGNED IN	STATUS	ACTIONS
Administrator	admin	Administrator	—	25 Sept, 10:21 am	ACTIVE	Edit
Boss	boss	Boss	BOSS	23 Sept, 10:44 pm	ACTIVE	Edit Delete
Employee 1	employee1	Employee	EMP-001	22 Aug, 11:12 am	ACTIVE	Edit Delete
Employee 2	employee2	Employee	EMP-002	Never	ACTIVE	Edit Delete
Employee 3	employee3	Employee	EMP-003	Never	ACTIVE	Edit Delete
Ramesh Patel	employee4	Employee	EMP-004	25 Aug, 1:10 pm	ACTIVE	Edit Delete
Order Book Employee	oemp	Employee	EMP-00	23 Sept, 10:45 pm	ACTIVE	Edit Delete

Figure 8 — Account management. The client adds an employee, edits a name, username or password, and hands those credentials over. Removal deletes an account with no history and archives one the audit log names; the administrator's own row offers no delete.

Operate Boss

BossChange passwordSign out

DashboardBoss App

Boss App

Orders, the ERP and chalan across the whole company.

DashboardView ordersERPChalan

View orders

Every order and where it has reached.

8

ERP

What the team has been given and what they have finished.

1

Chalan

Chalans against each order.

Arrives in Phase 5

Figure 9 — The Boss App landing screen. Chalan states the phase it arrives in rather than showing a zero.

Operate Boss

BossChange passwordSign out

DashboardBoss App

ERP

What the team has been given and what they have finished.

DashboardView ordersERPChalan

ERP is scaffolded, not finished. Below is the real data it will be built on, read only — tell me what this section should do and it gets built properly.

TASK-0005

Cut 50 flush doors for the Ahmedabad order

Employees 1

COMPLETED

Figure 10 — The ERP section: real task data, read-only, and labelled as awaiting the client's specification.

12 · AI-Assisted Development

This project is built with an AI coding assistant (Claude, used through Claude Code) writing the implementation under my direction. That is stated plainly because it is how the work happens, and because the division of responsibility matters more than the tool: the client owns the business requirements; I own the specification, the architecture, the engineering decisions, the review and the acceptance; the assistant produces code, migrations, tests and documentation against those decisions and my review. Nothing reaches the client that I have not run and checked.

12.1 How the work was organised

STAGE	WHAT HAPPENED
1 — Requirements into a specification	I turned the client's requirements into a written specification: modules, business rules, constraints (zero cost, self-hosted, no exposed database, no plain-text passwords, no hard-coded secrets), the three builds, the ten phases, and the security tests the system had to pass.
2 — Architecture before code	A standing instruction in the specification was that implementation could not begin until an architecture proposal had been presented and approved. The stack, the database design and the phase plan were reviewed and approved before the first line of code.
3 — Questions, not assumptions	Where the specification was silent, the rule was to stop and ask rather than guess. Decisions settled this way, and confirmed with the client, include: an order ships once rather than in parts; products are a hybrid of a catalogue and free text; quantities are a number plus a unit from a short list; order numbering never resets; a task may optionally link to an order; the wage rate is frozen when a task is created; a returned submission is kept as history.
4 — Phase-by-phase delivery	Each phase ends in a working system, its own tests, updated documentation and a single commit — eleven so far, each a reviewable unit of work the client can be shown.
5 — Testing, feedback, correction	I run each phase and direct changes from what I see, and from what the client reports back. Several features exist because the first attempt was rejected — for example, a temporary-password reset was replaced with administrator-controlled account editing, because in this business the owner sets credentials and hands them over in person.

12.2 What the assistant was used for, and what it was not

USED FOR	NOT USED FOR
Writing implementation code, migrations and tests to a stated design	Deciding business rules — these were asked of me and recorded
Explaining trade-offs so I could choose between them	Choosing the architecture without approval
Diagnosing failures I reported from running the application	Deciding when a phase was acceptable
Working around the constraints of the machine (no administrator rights, Windows shell behaviour)	Relaxing a security requirement to make something easier
Writing documentation and this report from the repository	Speaking to the client or agreeing scope on my behalf

EXAMPLE OF DIRECTION CHANGING THE PRODUCT

The first account-recovery implementation issued a temporary password and forced a change at next sign-in — a reasonable default, and wrong for this client, whose owner sets credentials and hands them over face to face. I had it replaced with an edit feature: the administrator sets the name, username and password. The reset flow was removed, the edit flow replaced it, and the audit action was renamed to match what actually happens. Later requirements in the same vein added employee creation, account deletion with archiving, a dedicated order-capture account for the client's order-book employee, and the Boss App section — each one arriving after the phase it belonged to was already "finished", which is the normal shape of this engagement.

12.3 Verification of AI-written code

- **Tests over claims.** 127 tests run against the real application and database; the security suite is written from the requirements in my specification rather than from the implementation.
- **Reading the diff.** Each phase is one commit, reviewed as a unit.
- **Using the application.** Every feature was exercised in the browser, on a desktop and at phone width, before it was accepted.
- **Preparing this report found a real defect.** Re-running the audit chain verification against the live database surfaced a verification bug (§19.1) and a flaky test fixture, which was fixed. Both are reported rather than omitted — the client is told what is wrong as well as what works.

13 · Challenges and Solutions

CHALLENGE	RESOLUTION
No administrator rights, no Docker	Node.js and PostgreSQL were installed as portable directories on a data drive, with the database cluster initialised by a script in the repository and bound to the loopback interface on a non-default port.
Windows PowerShell would not run the project	The default execution policy refuses local <code>.ps1</code> files, <code>&&</code> is not a valid separator in Windows PowerShell 5.1, and batch files with Unix line endings fail to parse. The fix was thin <code>.cmd</code> wrappers that invoke PowerShell with an explicit bypass, Git attributes pinning CRLF for <code>.cmd</code> , <code>.bat</code> and <code>.ps1</code> , and starting Node directly rather than through the npm shim.
Starting the database hung the terminal	<code>pg_ctl</code> inherited the terminal's output handles and never returned. It is now spawned with its output detached and the script polls for readiness instead.
Migrations failed in the shadow database	Hand-written migrations that grant privileges and create triggers were replayed into a shadow database that lacks the migration table, and guessed a sequence name. They were rewritten to ask PostgreSQL for the sequence by name and to guard every reference with an existence check.
The audit protection broke the test teardown	Deleting a test user was refused, correctly, because the log names them. Rather than weakening the constraint, the fixtures were changed to upsert and then archive — which is also what the application does for real accounts.
One package, two module systems	The shared contracts package is consumed by a CommonJS API and an ES module browser build. It is compiled twice with an exports map; the test runner maps it back to source so a change is picked up without a build step.
Refresh-token rotation versus concurrent queries	Four queries firing on one expired token presented the same rotating token four times, which the server correctly read as theft. The client now shares a single in-flight refresh.
Photographs returned 401	The browser issues <code></code> requests itself and cannot attach a bearer token. Instead of making files readable by cookie or by unguessable URL, a component fetches the bytes with the same credentials as every other call.
A vague error hid a stopped server	When the API was down, the development proxy answered 500 and the sign-in screen said "Something went wrong". Errors without the API's own body are now reported as <i>Server offline</i> , with tests covering it.
A test that passed until it did not	Fixture accounts derived their names from the clock and eventually collided with rows archived by earlier runs. Replaced with random suffixes while preparing this report; the suite is green again.

14 · Testing and Verification

API tests run against the real application — the real modules, the real guards, the real database — because the point is what the server does when the interface is bypassed entirely. Client tests mock only the network.

SUITE	TESTS	WHAT IT ESTABLISHES
<code>security/authorization.e2e-spec.ts</code>	31	Deny by default, permission enforcement, row-level scoping, account lifecycle, lockout, session revocation.
<code>orders/order-book.e2e-spec.ts</code>	24	Capture, correction, approval, rejection and cancellation; concurrent numbering; catalogue learning.
<code>tasks/tasks.e2e-spec.ts</code>	24	Assignment, evidence requirements, the correction loop, frozen earnings, cross-employee isolation.
<code>ingestion/message-parser.spec.ts</code>	25	Parsing of real-world message shapes, including ones it cannot read.
<code>audit/chain.e2e-spec.ts</code>	4	Append-only enforcement and hash-chain continuity.
<code>web: App.test.tsx</code>	9	Role-based landing, permission-filtered navigation, redirects for URLs typed directly.
<code>web: format.test.ts</code> , <code>api-client.test.ts</code>	10	Formatting helpers; offline and error-message behaviour of the API client.
Total	127	108 API (5 suites, 22.2 s), 19 web (3 files). All passing on 25 September 2026.

14.1 The security requirements, and the tests that answer them

REQUIREMENT FROM THE SPECIFICATION	HOW IT IS TESTED
Employee 1 cannot access employee 2's data	Signed in as one employee, requests for another's task and for another's dashboard are refused; a request for another's task returns 404, indistinguishable from a task that does not exist.
An employee cannot reach administrator functions through the API	Every administrative route is called with an employee's token and must be refused — and the refusal must appear in the audit log.
An incorrect order number cannot break the chain	Unknown, malformed and duplicate order references are rejected without side effects; twelve simultaneous approvals produce twelve distinct numbers.
The audit log cannot be altered	Update, delete and truncate attempts against <code>audit_logs</code> are asserted to fail, and the hash chain is verified across the rows a test writes.

14.2 Other verification

- `npm run typecheck` passes across all three packages.
- The application was exercised in a browser at desktop width and at 375 px.
- Live database state was inspected directly while preparing this report: 2,900 audit records (291 of them refusals), 650 captured messages, 8 orders, 47 stored photographs.

15 · Security, Reliability and Data Integrity

CONCERN	MEASURE
Password storage	Argon2id at the OWASP minimum profile (19 MiB, two passes). Passwords are never logged, never written to the audit record, and never returned.
Credential stuffing	Per-account lockout after a configured number of failures; an unknown username is verified against a decoy hash so timing and wording do not reveal whether it exists.
Session handling	Short-lived access token held in memory; refresh token stored hashed, returned in an <code>httpOnly</code> , <code>SameSite=strict</code> cookie, rotated on every use, with re-use revoking the session.
Immediate revocation	Sessions are re-read from the database on every request, so disabling an account takes effect at once.
Authorization	Global deny-by-default guard, named permissions per route, and row-level scoping inside every service.
Database exposure	PostgreSQL listens on loopback only. The API connects as a role that owns nothing and cannot read the migration history.
Record integrity	Append-only privileges plus triggers on <code>audit_logs</code> ; SHA-256 hash chain; refusals recorded as well as successes.
File safety	Uploads are restricted by type and size, re-encoded (which removes embedded metadata, GPS included), stored under a content hash, and served only through a permission check.
Transport surface	Helmet headers, a CORS allowlist, a request rate limit, and a single multipart file per request.
Secrets	Generated by a script into an environment file that is never committed; the API refuses to start if a secret is missing or too short.
Honest failure	When the API cannot be reached the client says the server is offline; there is no local fallback copy of the data that could diverge from the server.

16 · Deployment and Usage

16.1 Running it

One command starts the database, the API and the web client, and opens the browser; another stops everything, including the database.

```
start.cmd          // database + API + web client, then opens the browser
stop.cmd           // stops all three and releases the ports

npm test           // 127 tests across API and web
npm run typecheck  // all three packages
npm run db:status  // is PostgreSQL up?
```

COMPONENT	WHERE IT RUNS
PostgreSQL 17	Portable installation, cluster on a data drive, loopback interface, non-default port.
API	Compiled output on 127.0.0.1:3001 under /api/v1 ; the start script rebuilds only when sources are newer than the build.
Web client	Vite on port 5173, proxying /api to the API so the browser sees one origin.
Files	A local directory outside version control, addressed by content hash.
Process control	Process identifiers and logs are written to a working directory; the stop script kills by recorded identifier and then by port, so a stray process cannot hold a port.

NOT YET PRODUCTION DEPLOYMENT

Build 1 runs on the developer's machine for one user at a time. Making the home computer a server (Build 3) still requires TLS, a reverse proxy, automated backups, registration as a Windows service, and an accepted restore procedure. None of those are done, and the system should not be exposed to the internet until they are.

17 · Role and Contribution

This is a client project, and I am the person delivering it: analyst, architect and reviewer, responsible to the client for what the system does and whether it is fit to use, with an AI assistant producing the implementation to my direction (§12).

CONTRIBUTION	EVIDENCE
Requirements analysis and specification	I took the client's requirements and wrote the project specification: the three modules, the three builds, the ten phases, the constraints (zero cost, self-hosted, no exposed database, no plain-text passwords, no hard-coded secrets) and the security tests the system must pass.
Domain rules	Questions raised and settled with the client rather than assumed, then recorded and encoded: one dispatch per order; hybrid product catalogue; quantity as a number plus a unit; order numbers that never reset; five fixed chain stages; optional task-to-order linkage; wage rate frozen at task creation; returned submissions kept as history.
Architecture	I required an architecture proposal before implementation began, and chose the stack, the database design and the phase plan against the client's constraints — one ordinary PC, no budget, no exposed database.
Change requests during the build	New client requirements folded into the delivered phases: self-service password change; administrator-controlled account editing in place of temporary-password reset; employee creation; account deletion with archiving; a dedicated order-capture account; the Boss App and its three sections. Each was specified, implemented, tested and documented without disturbing what was already accepted.
Testing and acceptance	I run each phase before it goes to the client, report defects from real use — a failing sign-in, an account seeing a section it should not, a confusing error message — and require them fixed before moving on.
Environment and delivery	I worked to the client's operating constraints — Windows without administrator rights, a modest machine, no paid services — which shaped the toolchain, the hashing parameters and the image pipeline, and wrote the one-command start and stop scripts and the documentation the client uses to run it.

Eleven commits so far are under my authorship, each an approved phase or feature, with the AI assistant credited as co-author on the commits it wrote. The engagement continues: the next phases begin when the client settles the details they depend on.

18 · Verified Metrics

All figures below were measured on 25 September 2026 from the repository, the test runner and the running database. They describe work in progress, not a completed delivery.

17,497 LINES TRACKED	127 TESTS PASSING	37 ENDPOINTS	2,900 AUDIT RECORDS
--	---	------------------------------------	---

MEASURE	VALUE	SOURCE
Tracked files (excluding lockfile)	134	<code>git ls-files</code>
Lines of code, all tracked files	17,497	API 5,458 · web 5,740 · tests 1,838 · Prisma 1,922 · contracts 997 · scripts 252
Commits	11	One per phase or approved feature, 21 Aug 2026 onwards
Database tables · migrations	22 · 7	Prisma schema and migration directory
HTTP endpoints	37	Controller decorators
Permissions · roles	43 · 3	Shared contracts package; 59 role-permission rows seeded
Tests passing	127	108 API in 22.2 s, 19 web
Typecheck	pass	<code>npm run typecheck</code> , all packages

18.1 Exercise of the system (development database)

RECORD	COUNT	MEANING
Audit records	2,900	2,609 successful actions, 291 recorded refusals
Captured messages	650	Through the same endpoint a WhatsApp provider will use
Orders · order lines	8 · 14	Approved orders carry issued numbers; pending ones do not
Tasks · submissions	1 · 2	Includes a returned submission kept as history
Stored photographs	47	Re-encoded and content-addressed
Accounts (active)	8	Plus archived test fixtures, which the record still names

These are development figures from building and testing the system, not production usage.

19 · Limitations

19.1 Known defect: chain verification reports false mismatches

FOUND WHILE PREPARING THIS REPORT

Each audit record is hashed over its own contents, including the two JSON columns that hold the before-and-after values of a change. PostgreSQL's `jsonb` type does not preserve the key order of the document it was given, so when a row is read back and re-hashed for verification, the serialised form can differ from the one hashed at write time — and the row is reported as altered when it is not.

Measured on the development database on 25 September 2026: 2,900 records, of which 1,305 carry a JSON payload; 707 of those re-hash differently. Records without a JSON payload verify correctly, and **every record's stored link to its predecessor is intact**, so the chain itself is continuous — the fault is in how verification re-derives the hash, not in the data.

Fix: hash a canonical serialisation of the JSON (sorted keys), or store the exact text that was hashed alongside the row, and verify against that. Until it is fixed, the Phase 6 integrity scan cannot be relied on for rows carrying JSON. The append-only protections — database privileges and triggers — are unaffected.

19.2 Scope not yet built

This is an in-progress engagement, so the list below is work outstanding rather than work abandoned. Items marked as awaiting detail are blocked on decisions only the client can make.

MISSING	CONSEQUENCE
Traceability chain (Phase 5)	The module the specification treats as central — chalan, manufacturing, sales chalan, dispatch photograph, gate pass — does not exist yet. Field-level rules for each stage are still to be agreed with the client.
Wages and reports (Phase 7)	No wage calculation, no monthly statement, no export. The inputs exist: accepted submissions and the rate frozen on each task.
Integrity scan and its screen (Phase 6)	Verification is implemented as a routine but is not scheduled, and there is no administrator screen for it.
Boss App sections	View orders, ERP and Chalan render real data read-only and are explicitly labelled as scaffolding. The client has said they will specify what each section should do; the shell, the routes and the permissions are in place for when they do.
WhatsApp intake	The endpoint, parser, source records and de-duplication exist; no provider is connected, so messages are pasted in by a person.
Permission editing in the interface	Roles, grants and overrides are data and can be changed in the database, but there is no screen for editing them yet.
Android application (Build 2), home server (Build 3)	Not started. Build 3 additionally requires TLS, a reverse proxy, backups and service registration.

19.3 Operational limitations

- **Single machine, single user in practice.** Build 1 has not been load-tested and has no connection pooling tuned for concurrent users.
- **No automated backups.** A backup directory exists; the policy and schedule are part of Phase 10.
- **No HTTPS.** Traffic is plain HTTP on the loopback interface, which is acceptable only because nothing is exposed beyond the machine.

- **Password recovery for the administrator requires database access.** There is no second administrator account and no recovery flow; if the only administrator password is lost, a password must be set directly against the database.
- **No browser end-to-end tests.** Client testing is component-level plus manual use; the API is where the automated depth is.
- **Development data is present.** The working database contains seeded and test records; a production installation would start from the seed alone.
- **Accessibility has not been audited.** Tap-target sizes, contrast and keyboard paths were considered during implementation but no formal pass has been made.

20 · Future Improvements

ORDER	IMPROVEMENT	WHY IT COMES WHERE IT DOES
1	Fix hash canonicalisation and re-verify the chain	Integrity verification is the claim the whole audit design rests on; it should be sound before more records are written against it (§19.1).
2	Phase 5 — the traceability chain	The module the business most needs, and the reason order numbering and file evidence were built the way they were.
3	Phase 6 — scheduled integrity scan and mismatch detection	Duplicate photographs, stage contradictions and chain breaks become visible rather than discoverable only on inspection.
4	Phase 7 — wages and monthly reports	Turns recorded work into the number the business actually acts on each month.
5	Boss App sections, once the client specifies them	The shell and permissions exist; only the behaviour is outstanding.
6	Build 3 hardening: TLS, reverse proxy, automated backups, service registration, restore drill	Required before the system is reachable from outside the machine.
7	Build 2 — Flutter Android client	Employees submit work from the factory floor; the API and permissions are already shaped for it.
8	WhatsApp provider integration	Drops into the existing intake seam without changing the Order Book.
9	Role and permission editing in the interface	Makes the "policy is data" design usable by the owner rather than by someone with database access.
10	Second administrator account and a recovery procedure	Removes the single point of failure described in §19.3.
11	Browser end-to-end tests for the main paths	Capture → approve and assign → submit → review are the journeys worth protecting from regression.

21 · Conclusion

Operate replaces an informal, memory-based way of running the client's manufacturing business with a system where each order has one permanent number, each action has a named author, and each piece of finished work has evidence behind it. Four of ten phases are delivered: the database and security foundation, the application shell and dashboards, the Order Book with its intake seam, and the ERP task module with file storage. 127 automated tests, most of them against the real application and database, run green.

The work that carries the most engineering weight is not the screens. It is the decisions underneath them: an order number allocated in a single atomic statement so it can never be issued twice; an audit log that PostgreSQL itself refuses to let the application rewrite; authorization asked as two separate questions so that one employee cannot read another's work; permissions held as data so the owner can change who does what without a redeployment; and a parser that never discards what a customer actually wrote.

What remains is equally clear: the traceability chain and wages are specified but not built, the Boss App awaits the client's detail, and making the home computer a server needs work that has not been done. One real defect — verification re-deriving hashes for records that carry JSON — was found in preparing this report and is documented rather than hidden, with the fix identified. A system whose purpose is an honest record deserves an honest account of its own state.

The engagement is active. The phases already delivered are in the client's hands for testing, new requirements continue to arrive and are folded into the plan, and the next phases start as the decisions they depend on are made.

AT A GLANCE	
Complete	Authentication and authorization · audit · dashboards · Order Book and intake · ERP tasks and files · account management
Partial	Audit integrity scanning · Boss App sections · WhatsApp intake (seam only) · mismatch detection
Not started	Traceability chain · wages and reports · interface refinement pass · Build 1 acceptance · Builds 2 and 3

22 · Master Prompt

The following prompts recreate this project from nothing. The first is the primary version; the rest express the same brief in the structures preferred by other models. Each is self-contained.

22.1 Claude

<role>

You are a senior backend and full-stack engineer building a production system for a small manufacturer. You design before you code, you refuse to invent business rules, and you treat security and data integrity as requirements rather than features.

</role>

<context>

A client – a plywood and flush-door manufacturing business – runs on informal messages and paper. They need ONE application, "Operate", with three modules joined by a single permanent order number:

1. Order Book – customer orders from arrival to completion.
2. ERP – employee tasks, submissions with evidence, review, and monthly wages.
3. Order Traceability – a sequential chain per order: chalan, manufacturing, sales chalan, dispatch photograph, gate pass.

Delivery is in three builds over ONE backend and ONE database:

- Build 1 – web application (this task).
- Build 2 – Flutter Android client on the same API.
- Build 3 – the owner's home computer becomes the server; the database is never exposed, and clients show "Server offline" rather than falling back to any local copy.

Hard constraints: zero additional cost, free and open-source components only, self-hosted on one Windows PC with no administrator rights (so no Docker and no system services), 8 GB of memory.

</context>

<instructions>

1. Present an architecture proposal – stack, database design, module boundaries, phase plan – and WAIT for approval before writing implementation code.
2. Build in ten phases, each ending in a working system with its own tests, updated documentation and one commit: (1) database, authentication, authorization, audit; (2) application shell and dashboards; (3) Order Book and the intake seam; (4) ERP tasks and file storage; (5) traceability chain; (6) audit and integrity scanning; (7) wages and reports; (8) test pass; (9) interface refinement; (10) acceptance including backups and service registration.
3. Use a TypeScript monorepo: a NestJS API on Fastify with Prisma and PostgreSQL, a React + Vite client, and a shared contracts package holding Zod schemas, permission keys and role definitions, compiled for both CommonJS and ES modules.
4. Put every business rule in the backend. Clients hold none.
5. Authorization: a global deny-by-default authentication guard (routes are public only when explicitly marked); named permissions per route; and row-level scoping inside each service, so a caller without the matching ".any" permission sees only their own rows. Never branch on a role name – the administrator holds power by enumerated grant like everybody else.
6. Store roles, permissions, role grants and per-user overrides as database tables; a revoke beats a grant. Re-seeding may only add grants, never restore a revoked one.
7. Allocate order numbers with a single atomic statement against a counter row (INSERT ... ON CONFLICT DO UPDATE ... RETURNING). Never MAX()+1 or COUNT()+1. Gaps are acceptable; a reused number is not.
8. Make the audit log append-only in the database: the runtime role gets SELECT and INSERT on it and nothing else, and triggers refuse UPDATE, DELETE and TRUNCATE even from the owner. Chain each row with a SHA-256 over its contents plus the previous row's hash, written under a transaction-scoped advisory lock. Hash a canonical form of any JSON columns (sorted keys), or the verification will report false mismatches because jsonb does not preserve key order. Record refusals as well as successes. A failed audit write must not fail the business action.
9. Intake is a seam: POST /ingest/messages accepts a message with a source and an external id for de-duplication. A person pasting a message and a future WhatsApp provider use the same route, parser and approval. The parser must never reject a line – keep what it cannot read, keep the original text of every line forever, and state every guess it made.
10. A pending order and an approved order are the SAME row: approval assigns the number and moves the status. If the message identifies no customer, capture the order with none, say so, and refuse approval until a person chooses one. Create catalogue products on approval only.
11. Tasks carry three evidence flags – quantity, note, photograph – set by whoever creates the task and read by the server from the task, never from the request. There is no "submitted"

status: a task stays in progress while its submission waits, and "completed" means accepted by a second person. A returned submission is kept and the employee makes a new one. Freeze the wage rate onto the task when it is created.

12. Photographs: re-encode on upload (apply rotation, resize, drop all metadata including GPS), store under the SHA-256 of the processed bytes behind a storage interface, and serve only through a permission check. No publicly served directory.
13. Hash passwords with Argon2id at the OWASP minimum profile. Verify unknown usernames against a decoy hash. Lock an account after repeated failures. Keep the access token in memory only, the refresh token hashed in the database and delivered in an httpOnly, SameSite=strict cookie, rotated on every use, with re-use revoking the session; refresh single-flight on the client. Re-read the session from the database on every request so disabling an account is immediate.
14. Deleting an account must not erase the record: make the audit log's user reference ON DELETE RESTRICT, delete accounts with no history, and archive the rest.
15. Ship Windows-friendly control: start.cmd and stop.cmd wrappers (PowerShell refuses local .ps1 under the default execution policy, and "&&" is not a valid separator in Windows PowerShell 5.1), CRLF line endings pinned for .cmd/.bat/.ps1, PostgreSQL bound to loopback on a non-default port, and scripts that start, stop and report the database.
16. Write tests against the real application and the real database. At minimum prove: an employee cannot read another employee's data (and gets 404, not 403, for rows that are not theirs); an employee cannot reach administrator functions through the API, and the refusal is audited; concurrent approvals never produce a duplicate order number; the audit log cannot be updated, deleted or truncated; the hash chain verifies.

</instructions>

<constraints>

- Do not invent business rules. Where the specification is silent – how quantities are expressed, whether an order may ship in parts, what happens to a returned submission – STOP and ask.
- Zero cost: no cloud service, no managed database, no paid API, no CDN and no web fonts.
- The database is never exposed to the internet and is reached only by the API.
- Passwords are never stored in plain text and never written to any log, including the audit log.
- No hard-coded secrets, passwords or per-employee permissions. Secrets come from an environment file that is never committed; the API refuses to start without them.
- Interfaces must work at 375 px with tap targets of at least 44 px.
- Never report a module that does not exist as a zero; name it and the phase it arrives in.

</constraints>

<output_format>

Phase 0: an architecture proposal for approval – stack with reasons, database design, module boundaries, phase plan, and the questions you need answered before starting.

Then, per phase: the code, the migrations, the tests, updated README documentation, and a summary of what the phase established and what it deliberately did not do.

</output_format>

<final_check>

Before finishing a phase, verify: routes are closed unless marked public; every service narrows its query by caller; no role name is branched on; every state change writes an audit record; tests cover the four security requirements above; and nothing in the interface claims a capability the backend does not enforce.

</final_check>

22.2 OpenAI

Role: Senior full-stack engineer delivering "Operate", a self-hosted management system for a plywood and flush-door manufacturer: Order Book, ERP (employee tasks and wages) and Order Traceability, joined by one permanent order number, over one backend and one database.

Personality

Precise, sceptical of its own assumptions, and explicit about trade-offs. Asks rather than guesses when a business rule is missing. Writes for a reader who will maintain this in a year.

Goal

A working web application (Build 1) that an owner and their employees can run on one ordinary Windows PC, with the backend enforcing every rule, ready for a Flutter client (Build 2) and a home-server deployment (Build 3) without redesign.

Success criteria

- NestJS on Fastify + Prisma + PostgreSQL; React + Vite client; shared contracts package.
- Authentication is deny-by-default; permissions are database rows; services scope every query to the caller's own rows unless they hold the matching ".any" permission.
- Order numbers come from one atomic counter statement and are never reused.
- The audit log is append-only by database privilege and trigger, hash-chained, and records refusals as well as successes.
- Order intake keeps the customer's original text verbatim and never rejects a line.
- Tasks declare what evidence completing them requires; acceptance is a separate act by a second person; the wage rate is frozen when the task is created.
- Photographs are re-encoded, stripped of metadata, content-addressed and permission-checked.
- Tests run against the real application and database and prove cross-employee isolation, refusal of administrator functions to employees, unique numbering under concurrency, and an unalterable audit log.

Constraints

Zero additional cost, open-source only, self-hosted. No Docker or system services (Windows without administrator rights). Database on loopback only, reached solely by the API. Argon2id password hashing; no plain-text passwords anywhere, including logs. No hard-coded secrets or per-employee permissions. No CDN or web fonts. Interface usable at 375 px.

Output

An architecture proposal first, held for approval. Then ten phases, each delivering working code, migrations, tests, documentation and one commit, with a short statement of what the phase established and what it left out.

Stop rules

Stop and ask when a business rule is missing or ambiguous. Stop and report rather than weakening a security requirement to make something easier. If a phase's tests do not pass, fix them before starting the next phase; do not carry a known failure forward.

22.3 GPT-4.1

Role and Objective

Build "Operate", a self-hosted company management system for a plywood and flush-door manufacturer: one application, three modules (Order Book; ERP for employee tasks and wages; Order Traceability), one permanent order number linking them, one backend, one database. Deliver Build 1, the web application.

Instructions

- Present an architecture proposal and wait for approval before implementing.
- Use a TypeScript monorepo: NestJS on Fastify, Prisma, PostgreSQL, React + Vite, and a shared contracts package (Zod schemas, permission keys, role definitions) built for CommonJS and ESM.
- Keep every business rule in the backend; clients hold none.
- Deliver in ten phases, each with tests, documentation and one commit.

Sub-categories

Security: deny-by-default authentication guard; named permissions per route; row-level scoping in every service; Argon2id; account lockout; access token in memory; rotating refresh token, stored hashed, in an httpOnly SameSite=strict cookie, with re-use revoking the session; sessions re-read from the database on every request.

Data integrity: order numbers from one atomic counter statement; audit log append-only by database privilege and trigger; SHA-256 hash chain over canonicalised contents; refusals recorded; deleting an account restricted by the log's foreign key, with archiving instead.

Order Book: intake as an HTTP seam with source records and de-duplication; a parser that never rejects a line and keeps the original text; pending and approved are the same row; no customer means no approval; catalogue products created only on approval.

ERP: per-task evidence flags read from the task, not the request; no "submitted" status; returned submissions kept as history; wage rate frozen at creation; photographs re-encoded, metadata stripped, content-addressed, permission-checked.

Environment: Windows without administrator rights; portable Node.js and PostgreSQL; start.cmd and stop.cmd wrappers; CRLF pinned for .cmd/.bat/.ps1; database on loopback, non-default port.

Reasoning Steps

1. Restate the requirements and list every business rule the brief does not settle; ask about those before coding.
2. Propose the architecture and phase plan.
3. Implement one phase at a time.
4. After each phase, run the full test suite and the type check, then write the phase summary.

Output Format

Per phase: files changed with their contents, migrations, tests, documentation updates, the test run result, and a short "what this phase established / deliberately did not do" note.

Examples

Example 1 – Context: the brief does not say whether an order may be dispatched in parts.

Correct behaviour: stop, ask, and record the answer as a decision before implementing dispatch.

Final instructions

Never invent a business rule. Never weaken a security requirement for convenience. Never present an unbuilt module as a zero – name it and the phase it arrives in.

22.4 Codex

Objective

Implement "Operate": a self-hosted web application for a plywood and flush-door manufacturer, covering the Order Book, an ERP module for employee tasks and wages, and an order traceability chain, all keyed by one permanent order number.

Context

One backend and one database serve every client: a React web client now, a Flutter Android client later, and eventually a home computer acting as the server. The business runs on informal customer messages today; the system must keep the original message text and make every action attributable.

Current State

Nothing exists. The target machine runs Windows without administrator rights, with portable Node.js 24 and PostgreSQL 17 installed under a local directory.

Requirements

- TypeScript monorepo: apps/api (NestJS 11 on Fastify, Prisma 6), apps/web (React 19, Vite 6), packages/contracts (Zod schemas, permission keys, role definitions; CJS + ESM builds).
- Authentication: Argon2id, account lockout, JWT access token held in memory, rotating refresh token stored hashed and delivered in an httpOnly SameSite=strict cookie, session re-read per request.
- Authorization: global deny-by-default guard, named permissions per route stored as database rows with per-user overrides (revoke beats grant), and row-level scoping inside every service.
- Audit: append-only table (runtime role holds SELECT and INSERT only; triggers refuse UPDATE, DELETE and TRUNCATE), SHA-256 hash chain over canonicalised row contents under an advisory lock, refusals recorded, foreign key ON DELETE RESTRICT.
- Order Book: POST /ingest/messages intake seam with de-duplication, a forgiving parser that keeps original text, pending and approved as one row, atomic order-number allocation via INSERT ... ON CONFLICT DO UPDATE ... RETURNING, catalogue learning on approval only.
- ERP: tasks with quantity/note/photo evidence flags, start → submit → review with a correction loop, submissions retained, wage rate frozen at creation, photographs re-encoded and content-addressed.
- Windows control scripts: start.cmd and stop.cmd, database start/stop/status, environment bootstrap that generates secrets.

Constraints

- Free and open-source components only; no Docker, no cloud service, no CDN, no web fonts.
- PostgreSQL bound to loopback on a non-default port; only the API connects to it.
- Two database roles: an owner for migrations, and a runtime role that owns nothing.
- Secrets live in an uncommitted environment file; the API refuses to start without them.
- Do not branch on role names anywhere; do not hard-code permissions.
- .cmd, .bat and .ps1 files must use CRLF line endings.

Acceptance Criteria

- An employee cannot read another employee's tasks or dashboard; foreign rows return 404.
- An employee calling an administrator endpoint is refused and the refusal appears in the audit log.
- Twelve concurrent approvals produce twelve distinct order numbers.
- UPDATE, DELETE and TRUNCATE against the audit table fail; the hash chain verifies.
- An order whose message names no customer cannot be approved until one is chosen.
- A submission missing evidence the task requires is refused by the server.
- The client reports "Server offline" when the API cannot be reached, with no local data fallback.

Verification

- Run the full test suite (API against the real database, client in jsdom) and the type check.
- Start the application with start.cmd and exercise: capture a message, approve it, create a task, submit work with a photograph, return it, resubmit, accept it.
- Confirm the audit log contains both the successes and the refusals from those runs.

Deliverable

A running application with migrations, an idempotent seed, tests, control scripts and documentation that states which phases are complete and which are not.

22.5 Gemini

```
<role>
You are a senior full-stack engineer and data-integrity specialist.
</role>

<instructions>
1. Build "Operate", one application with three modules – Order Book, ERP (employee tasks and wages), Order Traceability – joined by a single permanent order number.
2. Propose the architecture and wait for approval before writing implementation code.
3. Use NestJS on Fastify with Prisma and PostgreSQL for the API, React with Vite for the client, and a shared contracts package for schemas, permission keys and roles.
4. Enforce every rule in the backend: deny-by-default authentication, named permissions stored as database rows, and row-level scoping in every service.
5. Allocate order numbers atomically from a counter row; never with MAX()+1 or COUNT()+1.
6. Make the audit log append-only in the database and hash-chain every row, canonicalising JSON before hashing; record refusals as well as successes.
7. Keep the customer's original message text forever; never reject a line the parser cannot read.
8. Let each task declare what evidence completes it; keep returned submissions; freeze the wage rate when the task is created.
9. Re-encode uploaded photographs, strip their metadata, store them under a content hash, and serve them only through a permission check.
10. Deliver in ten phases, each with tests against the real database, documentation and one commit.
</instructions>

<constraints>
- Zero additional cost; free and open-source only; self-hosted on one Windows PC with no administrator rights, so no Docker and no system services.
- The database is never exposed and is reached only by the API.
- No plain-text passwords, no hard-coded secrets, no permissions compiled into the code.
- Never invent a missing business rule – ask.
</constraints>

<output_format>
An architecture proposal first. Then, per phase: code, migrations, tests, documentation, and a statement of what the phase established and what it left undone.
</output_format>

<context>
The business currently records orders informally. Disputes about what was ordered, work counted twice and wages calculated from memory are the problems being solved. A later Flutter client and a home-server deployment must reuse this backend unchanged.
</context>

<task>
Deliver Build 1, the web application, phase by phase.
</task>

<final_instruction>
Prefer a refusal the user can understand over a convenience that hides what the system does not know. Never present an unbuilt module as an empty one.
</final_instruction>
```

22.6 Universal structure

ROLE

Senior full-stack engineer responsible for a self-hosted manufacturing management system and for the integrity of its records.

OBJECTIVE

Build "Operate" Build 1 – a web application with three modules (Order Book; ERP for employee tasks and wages; Order Traceability) over one backend and one database, keyed by a permanent order number, ready to serve a Flutter client and a home-server deployment without redesign.

CONTEXT

A plywood and flush-door manufacturer receives orders as free-form messages and records them late or not at all. Runs on one Windows PC, no administrator rights, 8 GB memory, zero budget.

REQUIREMENTS

1. TypeScript monorepo: NestJS on Fastify + Prisma + PostgreSQL; React + Vite; shared contracts.
2. All business rules in the backend.
3. Deny-by-default authentication; permissions as database rows; row-level scoping per service.
4. Atomic order numbering from a counter row; gaps allowed, reuse never.
5. Append-only, hash-chained audit log enforced by database privileges and triggers; refusals recorded; JSON canonicalised before hashing.
6. Intake as an HTTP seam with a forgiving parser that keeps the customer's original text.
7. Tasks with per-task evidence requirements, a correction loop that keeps history, and a wage rate frozen at creation.
8. Photographs re-encoded, metadata stripped, content-addressed, permission-checked.
9. Ten phases, each with tests, documentation and one commit.

CONSTRAINTS

Free and open-source only · no Docker or system services · database on loopback, reached only by the API · Argon2id, no plain-text passwords anywhere · no hard-coded secrets or permissions · no CDN or web fonts · usable at 375 px · never invent a business rule.

SUCCESS CRITERIA

Cross-employee isolation proven by tests (foreign rows return 404) · employees refused administrator endpoints, with the refusal audited · concurrent approvals never collide on a number · audit table resists UPDATE, DELETE and TRUNCATE and its chain verifies · an order with no customer cannot be approved · a submission missing required evidence is refused · the client says "Server offline" instead of falling back to local data.

OUTPUT FORMAT

Architecture proposal held for approval, then phase-by-phase delivery: code, migrations, tests, documentation, test results, and what each phase deliberately did not do.

VERIFICATION / STOP CONDITIONS

Run the full suite and type check after every phase. Stop and ask when a business rule is missing. Stop and report rather than weakening a security requirement. Do not begin a phase while the previous one has a failing test.